

SAPIENZA
UNIVERSITÀ DI ROMA

DEPARTMENT OF INGEGNERIA INFORMATICA, AUTOMATICA E GESTIONALE

Whole-body Motion Planning and Control for Underactuated Wheeled-Bipedal Robots

Professor:
Giuseppe Oriolo
Supervisor:
Francesco D'Orazio

Students:
Francesca Di Santo
Dario Finocchiaro
Arianna Navarra

Academic Year 2025/2026

Contents

1 Introduction **2**

2 Single Wheeled-Leg Kinematics Modeling **3**

2.1 Reference Frames and Modeling Assumptions 3

2.2 Definition of Joint Coordinates 4

2.3 Forward Kinematics and Jacobian derivation 5

3 Whole-Body motion planning **6**

3.1 Dynamic interpretation 6

3.2 Kinematic description 8

3.3 Whole-Body Jacobian 10

3.4 Control Architecture 12

3.5 Task space controller 13

3.6 Joint space controller 14

4 Implementation **15**

5 Simulation and results **20**

6 Conclusions **33**

References **34**

1 Introduction

Wheeled-bipedal robots represent a robotic architecture that combines high energy efficiency of wheeled systems with the versatility and terrain adaptability of bipedal robots. This class of systems has attracted significant attention in the robotics field due to its ability to perform agile and dynamic tasks quickly and efficiently.
However, wheeled-bipedal robots are unstable underactuated systems. Since the robot interacts with the ground through only two wheel contact points, the motion of the wheels and the posture of the upper body are strongly coupled. As a consequence, maintaining dynamic balance while executing motion commands becomes a challenging control problem.

Traditional control strategies for balancing wheeled robots, such as PID controllers, LQR-based methods and partial feedback linearization, struggle when full-body motion is considered. Recent works have introduced whole-body control (WBC) strategies to better handle this dynamic coupling. However, motion planning for underactuated wheeled-bipedal robots remains a non-trivial problem.

This project proposes a whole-body motion planning framework that considers the relationship between the horizontal posture and the acceleration of its center of mass (CoM), based on a Wheeled Inverted Pendulum Model (WIPM). The horizontal posture is included into the torso pose representation, allowing the derivation of a whole-body jacobian (WBJ) that defines the kinematic mapping between task space and joint space. This formulation enables the simultaneous control of torso pose, dynamic balance, forward velocity, and yaw rate.

The core part of the method lies in transforming the underactuated wheel torque control problem into a posture control problem: instead of directly commanding wheel rotation, the system controls the horizontal CoM displacement relative to the Zero Moment Point (ZMP), exploiting the relationship that links CoM acceleration to horizontal displacement. By planning the desired CoM trajectory in task space and mapping it to joint velocities via the WBJ matrix, the robot achieves coordinated whole-body motion while preserving dynamic stability.

The framework pipeline involves: implementing the kinematic and whole-body planning model, deriving and computing the Whole-Body Jacobian (WBJ), implementing task-space and joint-space control strategies. The proposed framework is then validated through numerical simulations carried out in MATLAB and Simulink, where the behavior of the robot under different motion tasks is analyzed.

2 Single Wheeled-Leg Kinematics Modeling

In the proposed control framework, the robot motion is not directly regulated through the rotation of the wheels, but through the control of the horizontal posture of the center of mass (CoM), which determines the CoM acceleration according to the Wheeled Inverted Pendulum Model (WIPM). Since the robot actuation is located at the joints of the legs, the control of the CoM posture must be achieved indirectly by commanding the leg joints.
For this reason, it is necessary to establish the relationship between the joint angles and the torso pose of the robot which is determined by the position of the hip joints.

2.1 Reference Frames and Modeling Assumptions

The wheeled-bipedal robot considered in this work is a floating-base mechanical system whose support with the environment is realized through two actuated wheels; a schematic version can be seen in figure 1. Three reference frames are introduced:

- Σ_W: inertial world frame;
- Σ_b: torso frame attached to the center of mass (CoM) of the torso;
- Σ_{wheel}: wheel frame located at the midpoint between the two axles of the wheels.

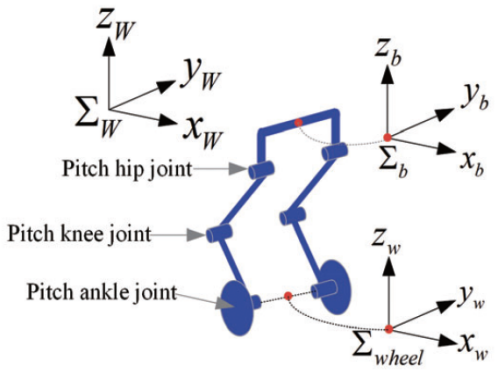


Figure 1: Wheeled-bipedal robot model and reference frames used in the kinematic formulation.

Because the two legs of the robot are symmetric, the kinematics of the entire system can be obtained by modeling a single leg and then applying the same formulation to both sides. The pose of the torso is subsequently reconstructed from the two hips.
Roll and yaw motions do not appear in the single-leg model and will only emerge when both legs and the torso are considered together. This planar assumption significantly simplifies the derivation while preserving the essential structure of the system.

The kinematic description of the robot is therefore derived starting from the model of a single wheeled leg. Each leg is composed of three pitch joints that connect the wheel, the crus and the thigh, forming a planar kinematic chain in the sagittal plane. In the extended configuration, the system can be interpreted as a floating-base system with the wheel acting as the base. Accordingly, the kinematic modeling of the single wheeled-leg is formulated with respect to the wheel frame. Moreover, since the dynamic stability of the robot depends on the relative position between the center of mass and the wheel-ground contact point, expressing the kinematics in Σ_{wheel} simplifies the kinematic description of the legs and the dynamic balance model.
The wheel frame Σ_{wheel} is defined such that:

- The x-axis points to the forward direction of the wheel.
- The z-axis points upward (opposite to gravity).
- The y-axis completes the right-handed coordinate system.
- The frame does not rotate with the wheel spinning.

2.2 Definition of Joint Coordinates

The joint coordinate vector for leg t, where t denotes the right or left leg, is defined as:

$$q_i^t = \begin{bmatrix} \theta_a^t & \theta_k^t & \theta_h^t \end{bmatrix}^T$$

where

- θ_a^t is the angle between the crus and the z-axis of the wheel frame. It is an angle measured with respect to the reference frame Σ_{wheel}, rather than a relative joint angle as in the classical Denavit-Hartenberg method.
- θ_k^t is the knee angle, defined as the angle between the thigh and the crus.
- θ_h^t is the hip angle, defined as the angle between the thigh and the z-axis of the hip frame at the initial configuration.

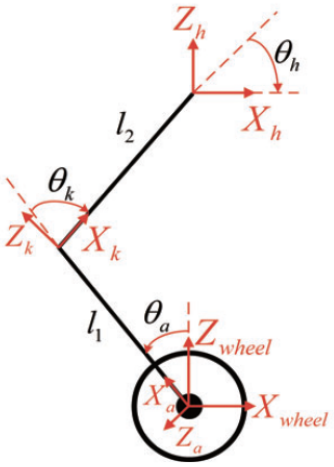


Figure 2: Kinematic structure of a single wheeled-leg showing the ankle, knee, and hip pitch joint.

Although unconventional compared to standard manipulator modeling, this approach is particularly convenient and greatly simplifies the computations in the following chapters.

2.3 Forward Kinematics and Jacobian derivation

The pose of the hip joint with respect to the wheel frame is now determined. The hip pose of for leg t is described by the vector:

$$q_h^t = \begin{bmatrix} x_h^t & z_h^t & \theta_{\text{pitch}}^t \end{bmatrix}^T$$

where x_h^t and z_h^t denote the hip position in the sagittal plane and θ_{pitch}^t represents the orientation of the thigh relative to the wheel frame.
Let l_1 and l_2 denote the lengths of the crus and thigh, respectively, assumed to be identical for both legs. The position of the hip is obtained as

$$x_h^t = -l_1 \sin\theta_a^t - l_2 \sin(\theta_a^t + \theta_k^t), \tag{1}$$
$$z_h^t = \quad l_1 \cos\theta_a^t + l_2 \cos(\theta_a^t + \theta_k^t), \tag{2}$$

and the pitch angle is

$$\theta_{\text{pitch}}^t = \theta_a^t + \theta_k^t + \theta_h^t \tag{3}$$

According to the kinematic model, these equations can be differentiated with respect to time to obtain:

$$\dot{x}_h^t = -(l_1 c_a^t + l_2 c_{ak}^t)\,\dot{\theta}_a^t - l_2 c_{ak}^t\,\dot{\theta}_k^t$$
$$\dot{z}_h^t = -(l_1 s_a^t + l_2 s_{ak}^t)\,\dot{\theta}_a^t - l_2 s_{ak}^t\,\dot{\theta}_k^t$$
$$\dot{\theta}_{\text{pitch}}^t = \dot{\theta}_a^t + \dot{\theta}_k^t + \dot{\theta}_h^t$$

Where s_a^t, s_k^t, c_a^t, c_k^t, s_{ak}^t, and c_{ak}^t represent $\sin\theta_a^t$, $\sin\theta_k^t$, $\cos\theta_a^t$, $\cos\theta_k^t$, $\sin(\theta_a^t + \theta_k^t)$, and $\cos(\theta_a^t + \theta_k^t)$, respectively. From this formula is obtained the relationship between joint velocities and the cartesian velocity of the hip in the sagittal plane:

$$\dot{q}_h^t = J_{\text{leg}}^t\,\dot{q}_j^t,$$

$$J_{\text{leg}}^t = \begin{bmatrix} -(l_1 c_a^t + l_2 c_{ak}^t) & -l_2 c_{ak}^t & 0 \\ -(l_1 s_a^t + l_2 s_{ak}^t) & -l_2 s_{ak}^t & 0 \\ 1 & 1 & 1 \end{bmatrix}.$$

3 Whole-Body motion planning

Having derived the kinematic model of a single wheeled leg, the next step consists in extending the formulation to the whole robot.
In the considered robot, the ankle actuator simultaneously affects the swing of the crus and the rotation of the wheel, as a consequence, the system is underactuated and the motion of the wheels cannot be controlled independently from the posture of the body. For this reason, the balance of the robot cannot be addressed as a purely geometric problem, instead a simplified dynamic model is introduced to describe the coupling between the motion of the center of mass and the wheels contact with the ground.
The objective of the whole-body formulation is therefore to establish a relationship between the desired torso motion and the joint-level actuation of the robot. This relationship will later allow the derivation of the Whole-Body Jacobian used for control.

3.1 Dynamic interpretation

In the sagittal plane, the robot can be approximated by a wheeled inverted pendulum, where a rigid body is supported above a rolling wheel-ground contact point.
To obtain the model, several simplifying assumptions are introduced:

- The analysis is restricted to motion in the sagittal plane.
- The wheels remain in contact with the ground without slipping.
- The height of the center of mass is assumed to remain approximately constant.
- The mass of the legs is negligible compared to that of the torso.

- The mass of the torso is assumed to be concentrated at its center of mass.

Taking moments about the wheel-ground contact point leads to the balance equation of the system:

$$m\ddot{x}_c z_c = mg(x_c - x_p)$$

where x_c, z_c denotes the positions of the center of mass, x_p denotes the ground contact point of the wheel, m is the total mass, and g is the gravitational acceleration. From this we can derive a model that is essentially a variation of the standard linear inverted pendulum model (LIPM):

$$\ddot{x}_c = \frac{g}{z_c}(x_c - x_p) = \frac{g}{z_c}x_{com} \tag{4}$$

where x_{com} is the horizontal displacement between the center of mass and the contact point.

This equation provides a direct interpretation of the balance behavior:

- If the center of mass is ahead of the contact point, the system accelerates forward.
- If it is behind, the system accelerates backward.
- If it is aligned vertically above the contact point, the acceleration is zero.

Therefore, the CoM acceleration can be modulated by controlling the horizontal posture. As shown in Fig. 3, the horizontal posture is related to the angle of the ankle joint. When the standing height remains approximately constant, the ankle joint angle is uniquely determined by the rotation of the wheel.

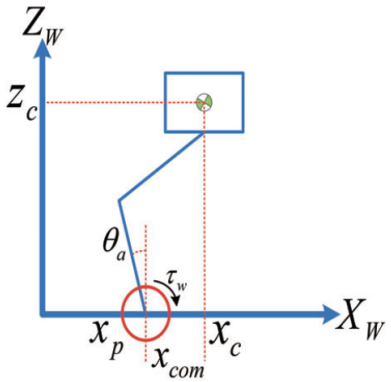


Figure 3: Wheeled-bipedal robot model in sagittal plane.

Since the wheels are torque-controlled actuators, the ankle joint angle can be used as an intermediate variable to regulate the wheel torques. The desired horizontal posture can be planned according to the dynamics of the model, which determines the desired ankle joint angle. The wheel torques are then adjusted so that the ankle joint tracks this reference value.

3.2 Kinematic description

To describe the whole-body motion of the robot, it is necessary to establish the relationship between the joint configuration of the legs and the pose of the torso. Since each wheel is connected to each crus through a single pitch joint, the system does not provide actuation in the lateral direction. As a consequence, the position of the torso along the y-axis cannot be actively controlled. For this reason, the task space is reduced by neglecting the lateral component, and the pose of the center of mass is described by a four-dimensional vector:

$$q_{com} = \begin{bmatrix} x_{com} & z_{com} & \gamma_{com} & \beta_{com} \end{bmatrix}^T$$

The three-dimensional poses of the left and right hips are then defined as:

$$p_h^l = \begin{bmatrix} x_h^l & z_h^l & \theta_{\text{pitch}}^l \end{bmatrix}^T, \quad p_h^r = \begin{bmatrix} x_h^r & z_h^r & \theta_{\text{pitch}}^r \end{bmatrix}^T$$

Where l represents the left leg and r represents the right leg. The geometric relationships between the motion of the hips and the motion of the torso are given by the following expressions.

In the sagittal projection, the center of mass lies midway between the two hip joints:

$$x_{com} = \frac{x_h^l + x_h^r}{2} \tag{5}$$

$$z_{com} = \frac{z_h^l + z_h^r}{2} \tag{6}$$

Differentiating:

$$\dot{x}_{com} = \frac{\dot{x}_h^l + \dot{x}_h^r}{2}$$

$$\dot{z}_{com} = \frac{\dot{z}_h^l + \dot{z}_h^r}{2}$$

The roll angle of the torso changes due to different heights of the left and right hips along the z direction:

$$z_h^r - z_h^l = l_b \tan(\gamma_{com})$$

Where l_b refers to the length of torso. To simplify the differential computation, the roll angle is replaced with the vertical difference between hips:

$$d = z_h^r - z_h^l = l_b \tan(\gamma_{com}) \tag{7}$$

Differentiating:

$$\dot{d} = \dot{z}_h^r - \dot{z}_h^l$$

The pitch angle is given by the average hip pitch:

$$\beta_{com} = \frac{\theta_{\text{pitch}}^l + \theta_{\text{pitch}}^r}{2} \tag{8}$$

Differentiating:

$$\dot{\beta}_{com} = \frac{\dot{\theta}_{\text{pitch}}^l + \dot{\theta}_{\text{pitch}}^r}{2}$$

Replacing the γ_{com} in the torso pose with d, the torso task space is:

$$q_{com} = \begin{bmatrix} x_{com} & z_{com} & d & \beta_{com} \end{bmatrix}^T$$

Without additional constraints, the system would be kinematically redundant, and the mapping between task-space velocities and joint velocities would not be unique. To avoid this redundancy and ensure a well-posed inversion of the whole-body Jacobian, two posture constraints are artificially introduced:

$$x_h^l = x_h^r$$
$$\theta_{\text{pitch}}^l = \theta_{\text{pitch}}^r$$

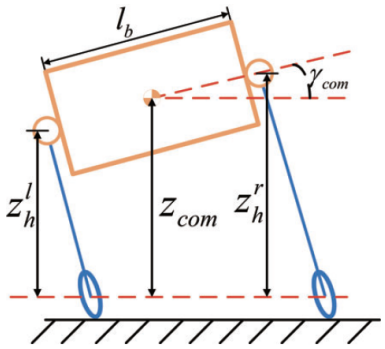


Figure 4: Front view of the wheeled-bipedal robot.

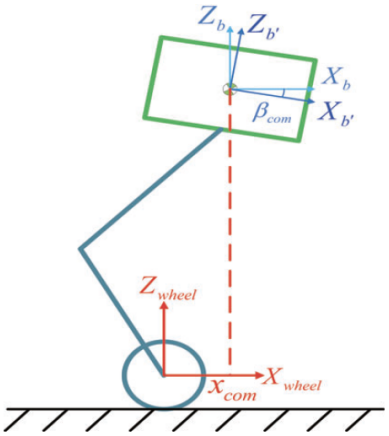


Figure 5: Left view of the wheeled-bipedal robot.

3.3 Whole-Body Jacobian

The derived equations can be reorganized to construct a whole-body velocity mapping that directly relates the joint velocities of both legs to the time derivative of the torso task variables. This mapping is referred to as the Whole-Body Jacobian (WBJ).

$$\dot{q}_{com} = \begin{bmatrix} \dot{x}_{com} \\ 0 \\ \dot{z}_{com} \\ \dot{d} \\ \dot{\beta}_{com} \\ 0 \end{bmatrix} = WBJ \begin{bmatrix} \dot{\theta}_a^l \\ \dot{\theta}_k^l \\ \dot{\theta}_h^l \\ \dot{\theta}_a^r \\ \dot{\theta}_k^r \\ \dot{\theta}_h^r \end{bmatrix} \tag{9}$$

so:

$$\dot{q}_{com} = WBJ \begin{bmatrix} \dot{q}_j^l \\ \dot{q}_j^r \end{bmatrix}$$

with

$$WBJ \in \mathbb{R}^{6\times 6}$$

Using the geometric relationships between hip motion and torso motion together with the single-leg kinematics, the Whole-Body Jacobian can be constructed row by row by differentiating the geometric definition of each torso variable or constraint.

$$\dot{x}_{com} = \frac{\dot{x}_h^l + \dot{x}_h^r}{2} = \begin{bmatrix} -\frac{l_1 c_a^l + l_2 c_{ak}^l}{2} & -\frac{l_2 c_{ak}^l}{2} & 0 & -\frac{l_1 c_a^r + l_2 c_{ak}^r}{2} & -\frac{l_2 c_{ak}^r}{2} & 0 \end{bmatrix} \begin{bmatrix} \dot{q}_j^l \\ \dot{q}_j^r \end{bmatrix}$$

$$0 = \dot{x}_h^l - \dot{x}_h^r = \begin{bmatrix} -(l_1 c_a^l + l_2 c_{ak}^l) & -l_2 c_{ak}^l & 0 & (l_1 c_a^r + l_2 c_{ak}^r) & l_2 c_{ak}^r & 0 \end{bmatrix} \begin{bmatrix} \dot{q}_j^l \\ \dot{q}_j^r \end{bmatrix}$$

$$\dot{z}_{com} = \frac{\dot{z}_h^l + \dot{z}_h^r}{2} = \begin{bmatrix} -\frac{l_1 s_a^l + l_2 s_{ak}^l}{2} & -\frac{l_2 s_{ak}^l}{2} & 0 & -\frac{l_1 s_a^r + l_2 s_{ak}^r}{2} & -\frac{l_2 s_{ak}^r}{2} & 0 \end{bmatrix} \begin{bmatrix} \dot{q}_j^l \\ \dot{q}_j^r \end{bmatrix}$$

$$\dot{d} = \dot{z}_h^r - \dot{z}_h^l = \begin{bmatrix} (l_1 s_a^l + l_2 s_{ak}^l) & l_2 s_{ak}^l & 0 & -(l_1 s_a^r + l_2 s_{ak}^r) & -l_2 s_{ak}^r & 0 \end{bmatrix} \begin{bmatrix} \dot{q}_j^l \\ \dot{q}_j^r \end{bmatrix}$$

$$\dot{\beta}_{com} = \frac{\dot{\theta}_{\text{pitch}}^l + \dot{\theta}_{\text{pitch}}^r}{2} = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \end{bmatrix} \begin{bmatrix} \dot{q}_j^l \\ \dot{q}_j^r \end{bmatrix}$$

$$0 = \dot{\theta}_{\text{pitch}}^l - \dot{\theta}_{\text{pitch}}^r = \begin{bmatrix} 1 & 1 & 1 & -1 & -1 & -1 \end{bmatrix} \begin{bmatrix} \dot{q}_j^l \\ \dot{q}_j^r \end{bmatrix}$$

Thus, assembling the WBJ it is obtained:

$$\text{WBJ} = \begin{bmatrix} -\frac{l_1 c_a^l + l_2 c_{ak}^l}{2} & -\frac{l_2 c_{ak}^l}{2} & 0 & -\frac{l_1 c_a^r + l_2 c_{ak}^r}{2} & -\frac{l_2 c_{ak}^r}{2} & 0 \\ -(l_1 c_a^l + l_2 c_{ak}^l) & -l_2 c_{ak}^l & 0 & (l_1 c_a^r + l_2 c_{ak}^r) & l_2 c_{ak}^r & 0 \\ -\frac{l_1 s_a^l + l_2 s_{ak}^l}{2} & -\frac{l_2 s_{ak}^l}{2} & 0 & -\frac{l_1 s_a^r + l_2 s_{ak}^r}{2} & -\frac{l_2 s_{ak}^r}{2} & 0 \\ (l_1 s_a^l + l_2 s_{ak}^l) & l_2 s_{ak}^l & 0 & -(l_1 s_a^r + l_2 s_{ak}^r) & -l_2 s_{ak}^r & 0 \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ 1 & 1 & 1 & -1 & -1 & -1 \end{bmatrix} \tag{10}$$

By introducing the additional constraints, the mapping becomes uniquely invertible. As a result, the Whole-Body Jacobian is full rank and the relationship between task-space velocities and joint velocities admits a unique solution, and this greatly simplifies motion planning and control.

Besides the four-dimensional torso pose in the wheel frame, the yaw rate $\dot{\alpha}_{com}$ in the world frame must also be planned to allow the robot to perform turning maneuvers. When the robot performs a turning maneuver, a lateral acceleration arises due to centrifugal effects. If the robot moves forward with velocity v and rotates with yaw rate $\dot{\alpha}$, the lateral acceleration is

$$a_{lat} = v\,\dot{\alpha}$$

Maintaining stability requires generating a roll angle that balances the centrifugal acceleration. The equilibrium of lateral forces gives:

$$\tan(\gamma_{com}) = \frac{v\,\dot{\alpha}}{g}$$

from which the balancing roll angle can be obtained as:

$$\hat{\gamma}_{com} = \arctan\left(\frac{\dot{\alpha}_{com}\,v_{com}}{g}\right)$$

and consequently the desired height difference between the hips is:

$$\hat{d} = l_b \tan(\hat{\gamma}_{com})$$

Therefore, lateral stability during turning is achieved by coordinating the vertical positions of the two legs, which generates a roll angle consistent with the dynamic requirements of the system.

3.4 Control Architecture

The control framework consists of two main components:

- Task-space controller;
- Joint-space controller.

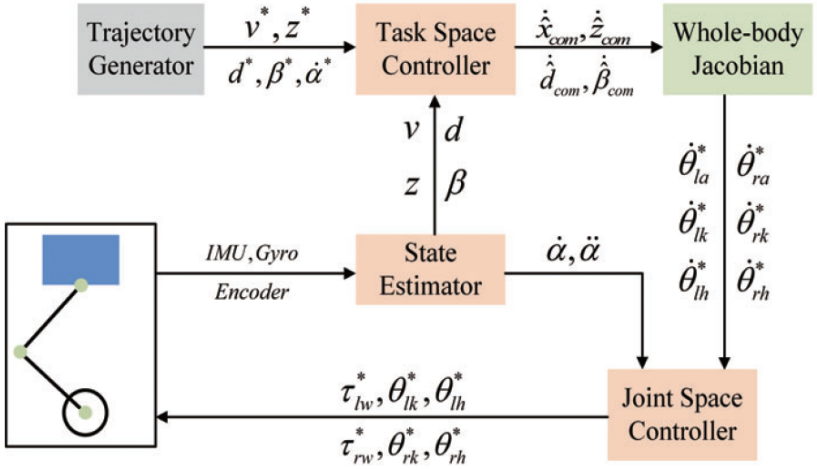


Figure 6: Overall control architecture of the wheeled-bipedal robot. The framework integrates trajectory generation, task-space control, Whole-Body Jacobian mapping, joint-space control, and state estimation.

The framework enables whole-body motion planning and control through the use of the Whole-Body Jacobian (WBJ).

The control architecture adopted in this work is illustrated in Fig. 6. The framework is organized in a hierarchical manner and integrates motion planning, task-space regulation, and joint-level actuation within a unified whole-body control strategy.
At the highest level, the trajectory generator provides the desired references for the robot motion, including forward velocity, standing height, leg height difference, torso pitch angle, and yaw rate. These references define the motion objectives in task space. The task-space controller compares these references with the estimated state of the robot and computes the desired torso velocities. Using the Whole-Body Jacobian, the resulting task-space commands are transformed into joint-space velocities, which are then executed by the joint-space controller. State feedback is provided by ODE integration in order to close the control loop.

3.5 Task space controller

The velocity and acceleration of the CoM can be controlled through the input of the horizontal posture in order to achieve speed tracking of the robot. The task-space variables are defined as:

$$\chi = \begin{bmatrix} v & 0 & z & d & \beta & 0 \end{bmatrix}^T,$$

where:

- v is the velocity of the center of mass with respect to the world frame,
- z is the standing height,
- d is the height difference between the two legs (which determines the roll),
- β is the torso pitch angle.

Additionally, the yaw rate α is considered to handle turning maneuvers. The task-space controller computes the desired rate of change of the torso configuration using a proportional-derivative structure:

$$\dot{\hat{q}}_{com} = K_P(\chi^* - \chi) + K_D(\dot{\chi}^* - \dot{\chi}) \tag{11}$$

where $\dot{\chi}$ and $\dot{\chi}^*$ represent the time derivatives of the current and desired task variables, respectively, while K_P and K_D are gain matrices.
Instead of directly commanding wheel rotation, the controller regulates the horizontal displacement between the CoM and the contact point, as previously discussed in the WIP-based modeling. By controlling this posture variable, the system indirectly regulates the forward acceleration and velocity of the robot.

3.6 Joint space controller

Once the desired torso velocity is computed, the Whole-Body Jacobian is used to determine the corresponding joint velocities.
The desired joint angular velocities are defined as:

$$\dot{q}_j^{l*} = \begin{bmatrix} \dot{\theta}_a^{l*} & \dot{\theta}_k^{l*} & \dot{\theta}_h^{l*} \end{bmatrix}^T, \qquad \dot{q}_j^{r*} = \begin{bmatrix} \dot{\theta}_a^{r*} & \dot{\theta}_k^{r*} & \dot{\theta}_h^{r*} \end{bmatrix}^T.$$

The desired joint velocities are then integrated to obtain the reference joint positions, expressed in discrete form as:

$$q_j^*(t+1) = q_j^*(t) + \dot{q}_j^* \Delta t.$$

For the hip and knee joints, classical position tracking control is applied. The ankle joints are mechanically coupled with the wheels and therefore cannot be directly position controlled in the same way as the other joints. They are instead controlled by regulating the wheel torques, which drive the ankle angle toward its reference value.
For this reason a proportional-derivative law is applied to the ankle angle tracking error:

$$\tau_w^l = k_p^l(\theta_a^{l*} - \theta_a^l) + k_d^l(\dot{\theta}_a^{l*} - \dot{\theta}_a^l) \tag{12}$$

$$\tau_w^r = k_p^r(\theta_a^{r*} - \theta_a^r) + k_d^r(\dot{\theta}_a^{r*} - \dot{\theta}_a^r) \tag{13}$$

where τ_w^l and τ_w^r are the torques applied to the left and right wheels, respectively, and $k_p^l, k_d^l, k_p^r, k_d^r$ are feedback gains.
For turning maneuvers, an additional torque component is introduced to create a differential torque between the left and right wheels. This generates the required yaw acceleration while preserving balance. The yaw-rate controller therefore modifies the wheel torques symmetrically but with opposite signs, producing controlled rotation of the robot.

$$\tilde{\tau} = \tilde{k}_p(\dot{\alpha}^* - \dot{\alpha}) + \tilde{k}_d(\ddot{\alpha}^* - \ddot{\alpha}) \tag{14}$$

$$\tau_w^{l*} = \tau_w^l + \tilde{\tau}, \qquad \tau_w^{r*} = \tau_w^r - \tilde{\tau} \tag{15}$$

where $\tilde{k}_p$ and $\tilde{k}_d$ are feedback gains, $\ddot{\alpha}^*$ is the desired yaw acceleration, and $\ddot{\alpha}$ is the measured yaw acceleration. Applying $\tilde{\tau}$ to the wheels generates a velocity difference between the left and right wheels, which allows the robot to perform turning maneuvers.

4 Implementation

This section describes the practical implementation of the proposed whole-body motion planning and control framework. The complete system has been implemented in MATLAB and Simulink and organized into several sections that reflect the theoretical formulation introduced previously.

Helper functions

To support the computations several auxiliary functions are implemented. These helper functions provide modular implementations of the single-leg forward kinematics, the reconstruction of torso task variables from the hip poses, and the computation of the Whole-Body Jacobian used to map task-space velocities into joint-space quantities.

- **Single Wheeled-Leg Kinematics Implementation:** The function `leg_fk.m` implements the forward kinematics of a single wheeled-leg in the sagittal plane (described in 2). Given the input joint vector:

  $$q_{leg} = [\theta_a \ \theta_k \ \theta_h]^T,$$

  According to 2 and 3, the function computes and returns the vector:

  $$[x_h, z_h, \theta_{pitch}]^T,$$

  which contains:

  - the hip position (x_h, z_h);
  - the hip pitch angle θ_{pitch}, obtained as the sum of the joint angles.

- **Task-Space Variable Reconstruction:** The function `task_from_hips.m` reconstructs the torso task-space variables starting from the hip poses of the left and right legs, following the geometric relationships derived in 3.2.

  Given the vectors:

  $$\text{hip}^l = [x_h^l, \ z_h^l, \ \theta_p^l]^T \quad \text{and} \quad \text{hip}^r = [x_h^r, \ z_h^r, \ \theta_p^r]^T,$$

  The function computes the task vector:

  $$\text{q}_{com} = \Big[x_{com} \ z_{com} \ d \ \beta_{com}\Big]^T.$$

  where:

  - x_{com} is the horizontal position of the CoM, computed according to (5);

- z_{com} is the average hip height, obtained from (6);
- d is the vertical difference between the right and left hip, defined in (7);
- β_{com} is the torso pitch angle, computed as shown in (8).

This function is called at each loop iteration to reconstruct the current task-space state used by the controller.

- **Whole-Body Jacobian Computation:** The function `compute_WBJ.m` constructs the Whole-Body Jacobian according to the formulation (10). At each iteration this function is used to evaluate the WBJ matrix from the current joint configuration vector:

$$q = \begin{bmatrix} q^l \\ q^r \end{bmatrix} = \begin{bmatrix} \theta_a^l \\ \theta_k^l \\ \theta_h^l \\ \theta_a^r \\ \theta_k^r \\ \theta_h^r \end{bmatrix}.$$

Main Simulation and Control Loop

The main MATLAB script implements the complete closed-loop simulation of the wheeled-bipedal robot. This section describes how the control architecture shown in figure 6 is translated into executable code and numerically integrated using the `ode45` solver.

Parameter Definition and Controller Gains

The geometric parameters of the robot are extracted from the mesh model and are defined as follows:

- Thigh length $l_1 = 0.27\,\mathrm{m}$
- Crus length $l_2 = 0.27\,\mathrm{m}$
- Wheel radius $r = 0.093\,\mathrm{m}$
- Distance between the legs $l_b = 1.2084\,\mathrm{m}$

The control gains used in the simulation are the task-space PD gains, the joint-space PD gains and the wheel torque PD gains. The gains reflect the hierarchical structure of the controller:

- Outer loop: task-space tracking;
- Inner loop: joint-space tracking;
- Low-level actuation: wheel torque control;

State Initialization

The complete simulation is implemented within the `ode45` integration loop. For this reason, the initial state of the system is defined through a state vector that collects all the variables required to describe the robot configuration and its dynamics.

$$x_0 = \begin{bmatrix} q^l & \dot{q}^l & q^r & \dot{q}^r & v & \dot{\alpha} & \omega^l & \omega^r & q^{l*} & q^{r*} \end{bmatrix}^T \tag{16}$$

The state vector includes the following quantities:

- joint positions and velocities of the left leg $(q^l, \dot{q}^l)$;
- joint positions and velocities of the right leg $(q^r, \dot{q}^r)$;
- forward velocity of the center of mass v;
- yaw rate $\dot{\alpha}$;
- angular velocities of the left and right wheels (ω^l, ω^r);
- desired joint reference positions for the left and right legs (q^{l*}, q^{r*}).

This state representation allows the ODE solver to integrate the evolution of both the mechanical states of the robot and the internal reference variables required by the control architecture.

Task Reference Generation

This block corresponds to the Trajectory Generator shown in figure 6. The desired task is defined through time-varying reference trajectories in the task space.
In particular, the following reference signals are defined:

- traj.chi_star(t): desired evolution of the task-space vector $\chi^*(t)$
- traj.dchi_star(t): time derivative of the desired task trajectory $\dot{\chi}^*(t)$
- traj.dalpha(t): desired yaw-rate trajectory $\dot{\alpha}^*(t)$
- traj.ddalpha(t): desired yaw angular acceleration $\ddot{\alpha}^*(t)$

Different task scenarios can be generated by modifying these reference trajectories. In particular, the reference signals were varied to evaluate different behaviors of the robot, such as forward velocity tracking and turning maneuvers. These experiments will be discussed in detail in the Results section.

Integration loop with `ode45`

The complete closed-loop dynamics of the wheeled-bipedal robot are numerically integrated using the MATLAB solver `ode45`. The simulation is executed through the following command:

$$[t, X] = \texttt{ode45}(@(t, x)\, \texttt{robot_ode}(t, x, p, \texttt{traj}),\ tspan,\ x_0,\ \texttt{opts}) \tag{17}$$

The solver `ode45` is a variable-step Runge-Kutta method of order $(4, 5)$ commonly used for the numerical integration of ordinary differential equations. It computes the evolution of the system state by integrating the nonlinear dynamical model:

$$\dot{x} = f(t, x) \tag{18}$$

where x is the state vector of the robot and $f(t, x)$ represents the closed-loop dynamics of the system.
The function `robot_ode` implements the full control pipeline and returns the time derivative of the state vector. In particular, it includes the following computational steps:

- Reconstruction of the task-space variables;
- Task-space control computation;
- Whole-Body Jacobian evaluation and inversion;
- Joint-space control;
- Wheel torque control.

At each integration step, the solver evaluates `robot_ode` using the current state of the system and updates the state vector according to the resulting state derivatives.

Control Loop Pipeline

At each integration step, the function `robot_ode` executes the following sequence of computations:

1. **State unpacking.** The current state vector is unpacked according to the structure defined in equation 16. This allows the following computation to use the current updated variables.
2. **Yaw rate computation.** The yaw rate is not directly included in the state vector. Instead, it is reconstructed from the angular velocities of the two wheels using the differential drive kinematic relationship:

$$\dot{\alpha} = \frac{r}{l_b}(\omega^l - \omega^r) \tag{19}$$

This formulation is adopted because, similarly to differential-drive robots, the turning motion of the wheeled-bipedal system is generated by the relative angular velocity of the two wheels mounted on the sides of the torso.

3. **Reference trajectory evaluation.** The desired task trajectories are evaluated at the current simulation time. In particular, the reference quantities $\chi^*(t)$, $\dot{\chi}^*(t)$, $\dot{\alpha}^*(t)$ and $\ddot{\alpha}^*(t)$ are obtained from the trajectory generator functions.

4. **Whole-Body Jacobian computation.** The Whole-Body Jacobian (WBJ) is computed from the current joint configuration and used later for the current $\dot{\chi}$ as well as the joint-space controller.

5. **Task-space state reconstruction.** The current task-space variables are reconstructed from the joint configuration of the two legs. First, the three-dimensional pose of each hip q_h is obtained by applying the forward kinematics function `leg_fk` to the joint vector of each leg q_j. The hip poses of the left and right legs are then used to reconstruct q_{com} through the helper function `task_from_hips`.

6. **Computation of χ and $\dot{\chi}$.** The task vector χ is computed from the reconstructed quantities obtained in Steps 1 and 5. The corresponding task-space velocity $\dot{\chi}$ is:

$$\dot{\chi} = \begin{bmatrix} \dot{v} & 0 & \dot{z} & \dot{d} & \dot{\beta} & 0 \end{bmatrix}^T.$$

Where the forward acceleration component is computed using the linear inverted pendulum equation 4 while $\dot{z}$, $\dot{d}$, $\dot{\beta}$ can be computed from the product:

$$\begin{bmatrix} \dot{x} & 0 & \dot{z} & \dot{d} & \dot{\beta} & 0 \end{bmatrix}^T = \text{WBJ} \begin{bmatrix} \dot{q}^l \\ \dot{q}^r \end{bmatrix}$$

7. **Task-space control computation.** Once the task vector χ and its derivative $\dot{\chi}$ are available, the task-space controller derived in 3.5 can be applied to compute the desired task velocity $\dot{\hat{q}}_{com}$ according to the proportional-derivative law in 11.

8. **Desired joint velocity computation.** Given the desired task velocity $\dot{\hat{q}}_{com}$, the joint velocities are computed by inverting the whole-body Jacobian mapping 9. The solution is obtained using the Damped Least-Squares (DLS) method in order to improve numerical robustness near singular configurations.

9. **Wheel torque computation.** Due to the underactuation the wheels are torque-controlled in order to regulate the ankle joint angle. The base torque

components 12 and 13 are computed using the tracking errors between the desired and current ankle angles and their derivatives. The additional correction term 14, introduced to generate turning motion, is then computed using current and desired yaw rate and yaw acceleration.

10. **Wheel angular acceleration.** As a simplifying assumption, the angular accelerations of the wheels are approximated from the computed relative torques.

11. **Joint-space position control.** To track the evolution of the leg joint positions and velocities, a proportional-derivative (PD) control law is used, based on the position and velocity errors.

$$\ddot{q}^l = K_{p,j}(q^{l*} - q^l) + K_{d,j}(\dot{q}^{l*} - \dot{q}^l)$$

$$\ddot{q}^r = K_{p,j}(q^{r*} - q^r) + K_{d,j}(\dot{q}^{r*} - \dot{q}^r)$$

12. **State derivative construction.** Finally, the derivative of the state vector is assembled using all the quantities computed during the loop. This derivative vector is returned to the ODE solver in order to integrate the system dynamics as in Eq. 18.

5 Simulation and results

The proposed numerical control loop was validated through different simulations in order to evaluate the behavior of the robot under different motion tasks. To provide a visual representation of the robot motion, the simulation results are also displayed in a Simulink environment. The visualization is based on the URDF model of the TITA robot together with the corresponding mesh files. Additional virtual joints were introduced to artificially reproduce the motion generated during the ODE-based simulation. These joints do not correspond to physical degrees of freedom, but are implemented in Simulink for visualization purposes, ensuring a consistent animation of the robot configuration with respect to the simulated trajectories.
Three simulation experiments were conducted to evaluate different aspects of the proposed control architecture:

- Velocity tracking task;
- Velocity and yaw tracking task;
- Slalom trajectory.

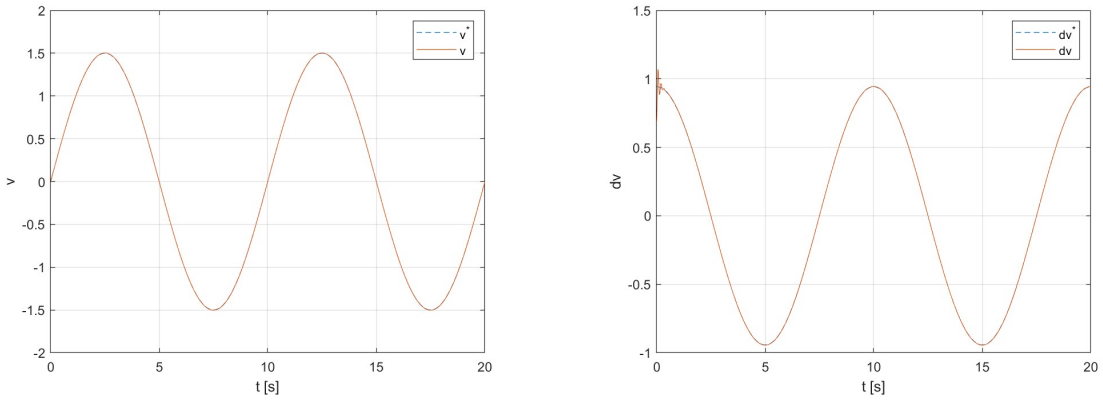
Velocity tracking task

In the first simulation, the control objective is to track a time-varying forward velocity of the CoM.
The desired task-space trajectories are defined as follows:

$$\chi^*(t) = \begin{bmatrix} 1.5\sin(0.2\pi t) & 0 & z_0 & d_0 & \beta_0 & 0 \end{bmatrix}^T, \quad \dot{\chi}^*(t) = \begin{bmatrix} 0.3\pi\cos(0.2\pi t) & 0 & 0 & 0 & 0 & 0 \end{bmatrix}^T$$

$$\dot{\alpha}(t) = 0, \quad \ddot{\alpha}(t) = 0$$

The robot is required to follow a sinusoidal velocity profile along the forward direction, while the remaining task variables are regulated around constant values to maintain balance. No turning motion is imposed. As shown in Fig. 7a, the CoM velocity and acceleration closely follows the desired sinusoidal reference.

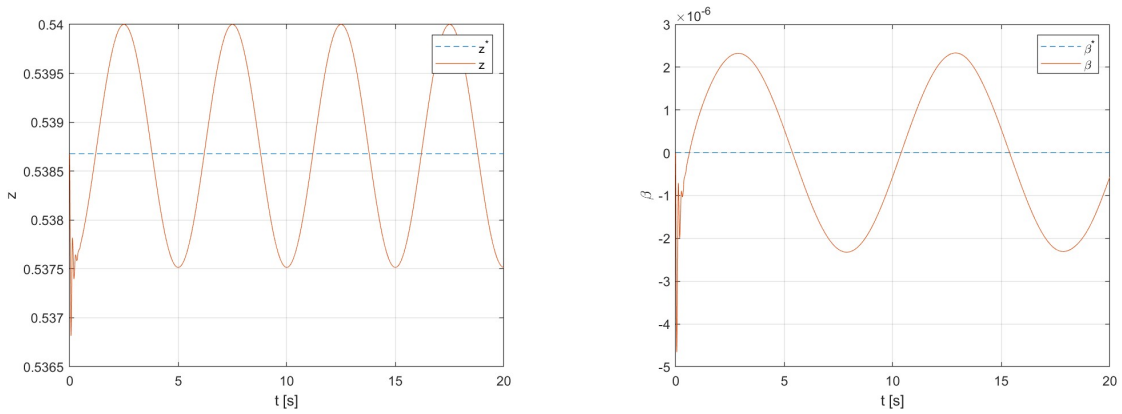


(a) CoM velocity tracking

(b) CoM acceleration

Figure 7: CoM velocity and corresponding acceleration.

The CoM height and torso pitch remain stable, confirming that balance is preserved throughout the motion. As shown in Fig. 8, the CoM height exhibits small oscillations around its nominal value. These oscillations are expected and are associated with the forward-backward motion of the robot: during acceleration phases, the robot slightly leans forward, leading to a temporary decrease in the CoM height, while it rises again as the posture is readjusted when the acceleration comes close to zero. Due to the periodic nature of the task, the opposite behavior is observed during negative acceleration phases, where the robot leans backward and the CoM height varies accordingly.
The torso pitch angle remains close to zero with only small variations.

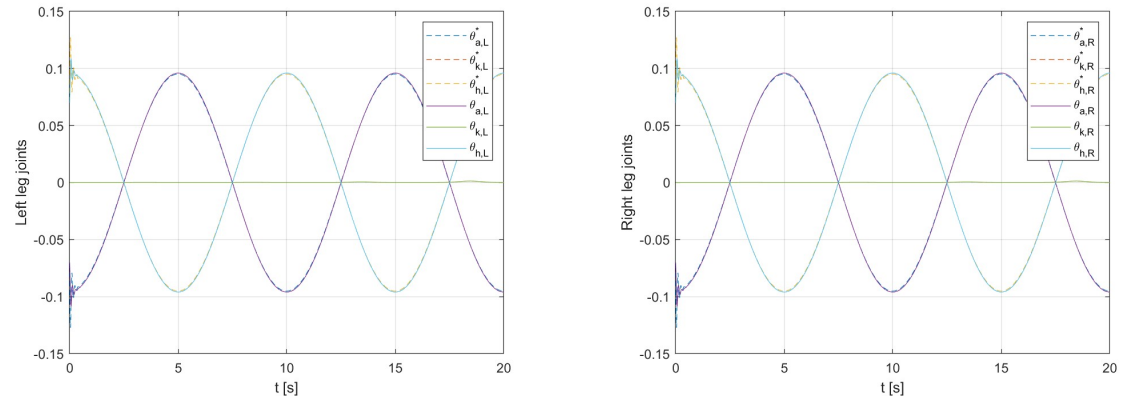


(a) CoM height (b) Torso pitch

Figure 8: CoM height and Torso pitch during velocity tracking task.

The evolution of the joint variables is reported in Fig. 9. The joint trajectories remain smooth, indicating that the motion is physically consistent and well distributed across the robot structure.

The observed behavior is consistent with the expected kinematic response: since the robot is required to follow a sinusoidal forward velocity while maintaining a constant posture, the leg joints exhibit periodic and coordinated oscillations. In particular, the ankle and hip angles vary smoothly to regulate the horizontal motion of the CoM, which directly influences the forward acceleration according to the WIPM model. Moreover, the symmetry of the task implies that no differential motion between the two legs is required: as a result, the knee joints of the left and right legs exhibit identical behavior and remain approximately at rest, since the forward-backward leaning motion is achieved through coordinated variations of the ankle and hip angles.

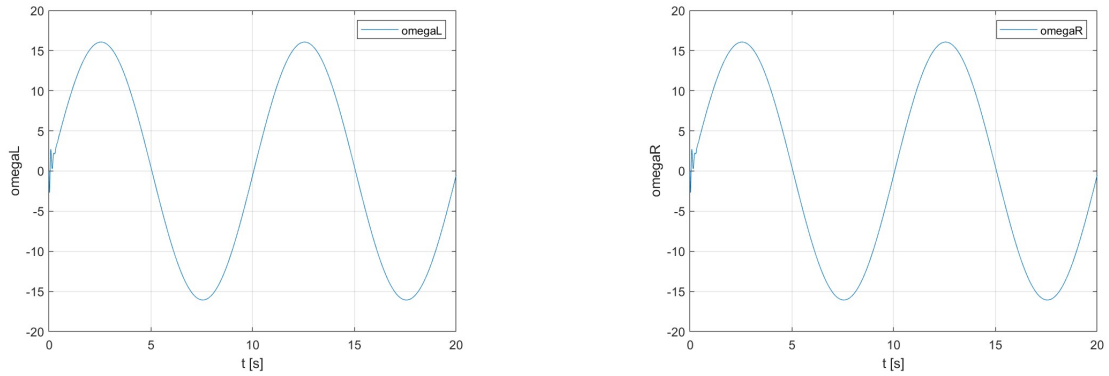


(a) Left leg joints (b) Right leg joints

Figure 9: Joint trajectories during velocity tracking task.

The wheel angular velocities are shown in Fig. 10. As expected, the two wheels exhibit identical behavior due to the absence of yaw motion. The angular velocities follow a

sinusoidal profile consistent with the forward velocity tracking task.



(a) Left wheel angular velocity

(b) Right wheel angular velocity

Figure 10: Wheel angular velocities during velocity tracking task.

The following figures 11 show the results obtained from the ODE-based numerical simulation, visualized in Simulink using the reconstructed robot model.



Figure 11: Sequence of frames showing the robot motion during the velocity tracking task.

Velocity and yaw tracking task

In the second simulation, the control objective is to track both a forward velocity and a time-varying yaw rate of the robot. This experiment evaluates the capability of the control framework to generate turning motion while maintaining dynamic balance. The desired trajectories are defined as follows:

$$\dot{\alpha}(t) = \begin{cases} 0.3\,t & t \leq 5 \\ 1.5 & t > 5 \end{cases}, \quad \ddot{\alpha}(t) = \begin{cases} 0.3 & t \leq 5 \\ 0 & t > 5 \end{cases},$$

$$\chi^*(t) = \begin{bmatrix} 1.5 & 0 & z_0 & l_b \dfrac{1.5\,\dot{\alpha}(t)}{9.81} & \beta_0 & 0 \end{bmatrix}^T,$$

$$\dot{\chi}^*(t) = \begin{bmatrix} 0 & 0 & 0 & l_b \dfrac{1.5}{g}\ddot{\alpha}(t) & 0 & 0 \end{bmatrix}^T$$

As shown in Fig. 12, the resulting trajectory of the robot is illustrated in the plane.

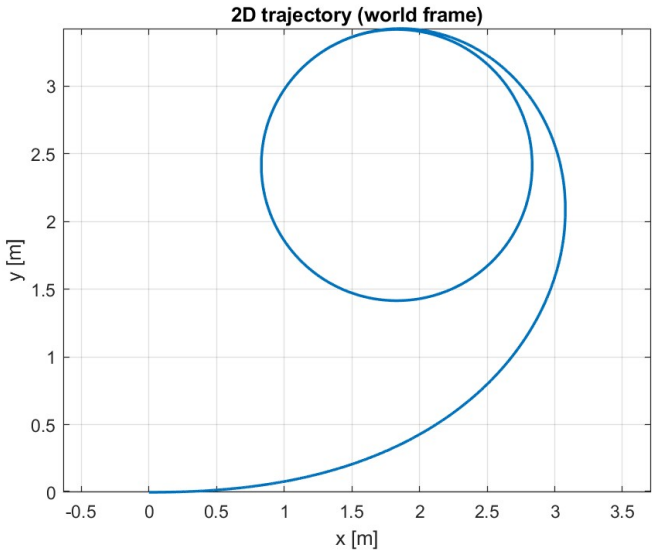


Figure 12: 2D trajectory of the robot during turning motion.

In Fig. 13, the forward velocity quickly converges to the desired constant value of 1.5 m/s, with a smooth response and very small error.

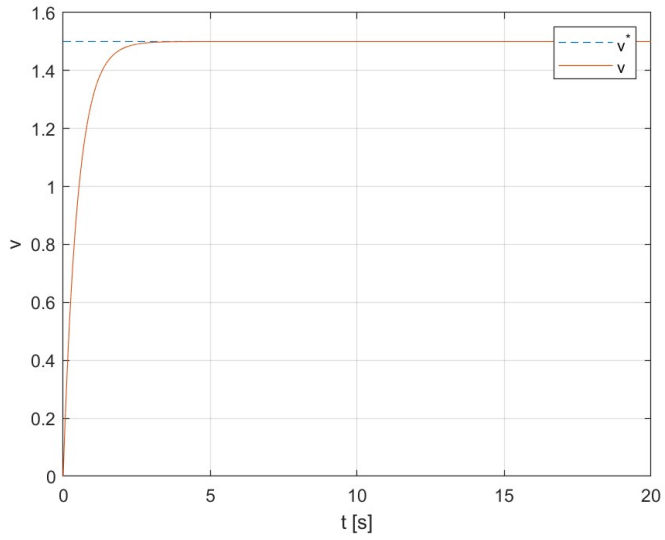
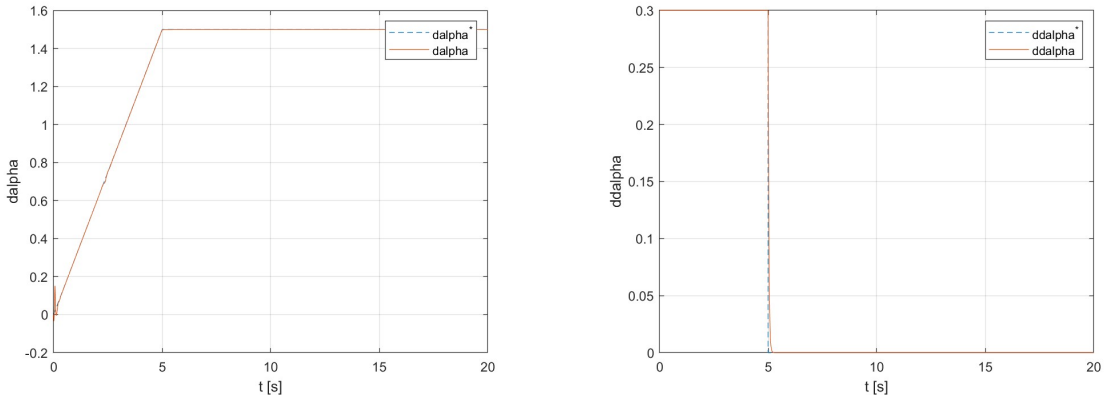


Figure 13: Forward velocity tracking.

The yaw rate follows the reference profile accurately (Fig.14a): it increases linearly during the initial phase and reaches the desired constant value after approximately 5 seconds.
The yaw acceleration remains bounded and exhibits the expected profile, with a constant value during the acceleration phase and zero afterward 14b.

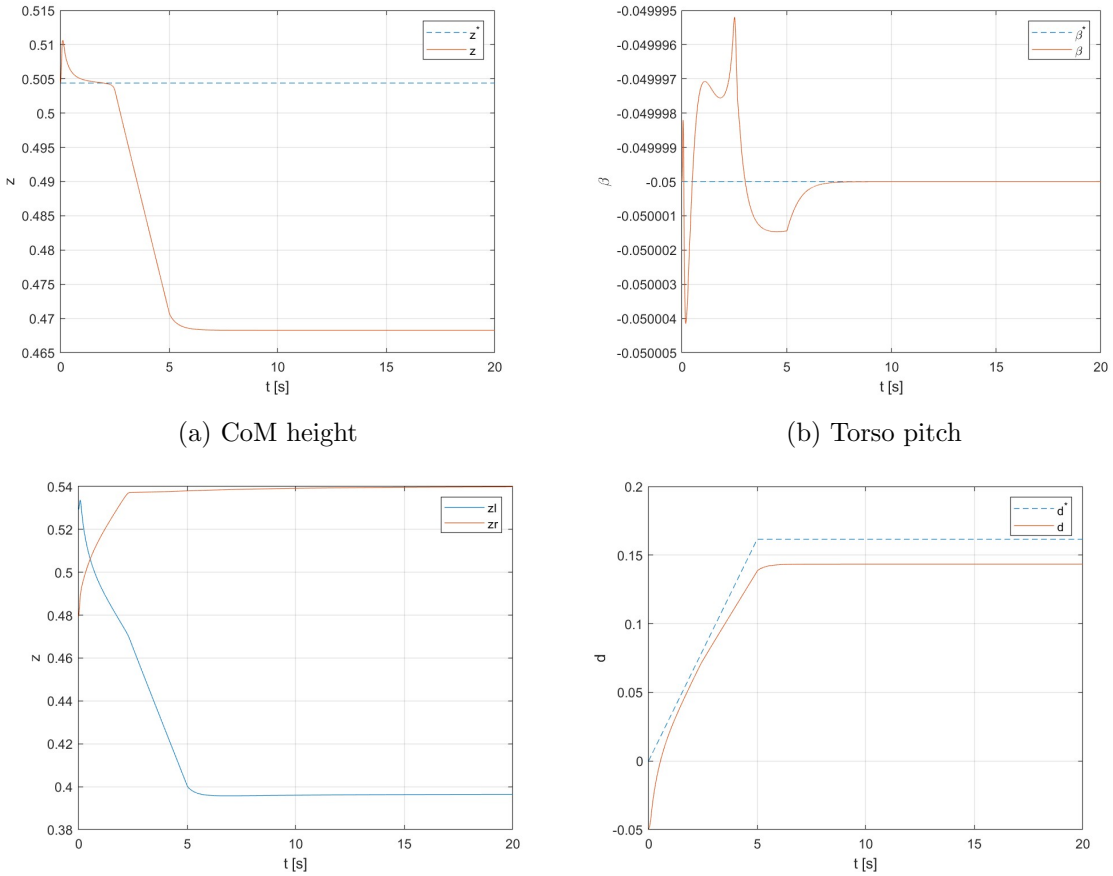


(a) Yaw rate

(b) Yaw acceleration

Figure 14: Tracking performance of forward velocity, yaw rate, and yaw acceleration.

The turning behavior is achieved through differential actuation of the two wheels, which produces a controlled rotation around the vertical axis. This results in a coordinated motion where the robot simultaneously translates and rotates.
The behavior of the CoM height, torso pitch angle, and leg configuration parameter during the turning motion is shown in Fig. 15.
As the robot progressively enters the turning motion, the CoM height decreases (Fig. 15a). This reduction is consistent with the change in posture required for turning: as the robot leans on one side to follow the curved trajectory, the vertical position of the CoM is slightly lowered.



(a) CoM height

(b) Torso pitch

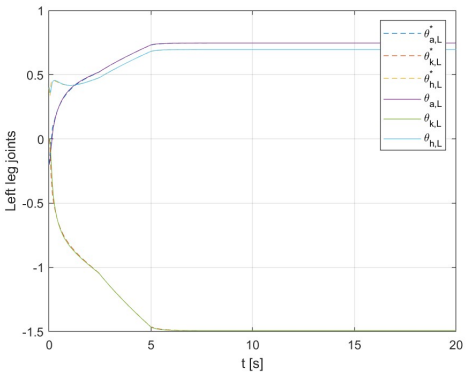
(c) Leg heights

(d) Leg configuration

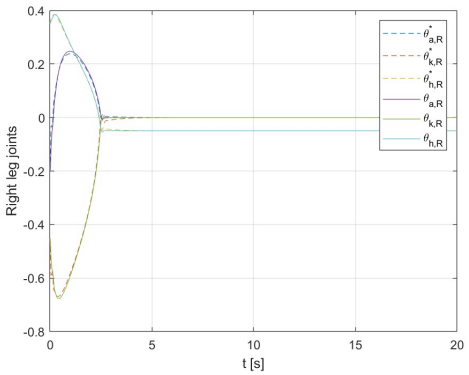
Figure 15: CoM height, torso pitch and leg configuration during turning motion.

In this case, since the robot turns to the left, Fig. 15c shows that the left leg lowers while the right leg rises. This coordinated motion generates the observed value of d, which is therefore a direct consequence of the vertical displacement between the two legs. In particular, d represents the vertical difference between the two legs and directly determines the roll angle of the torso. As the yaw rate increases, d adjusts to generate the required roll angle that compensates for the lateral acceleration induced by turning, in agreement with the equilibrium condition derived from the WIP-based model.

The joint trajectories are shown in Fig. 16. The joint motions remain smooth and continuous, with a coordinated but asymmetric behavior between the left and right legs. This asymmetry is a direct consequence of the turning motion: the two legs no longer move identically, but instead exhibit differentiated trajectories to generate the required roll configuration and support the lateral dynamics. In particular, as the robot begins to turn, one leg effectively extends (the right one) while the other shortens (the left one), producing the vertical leg difference encoded by d.



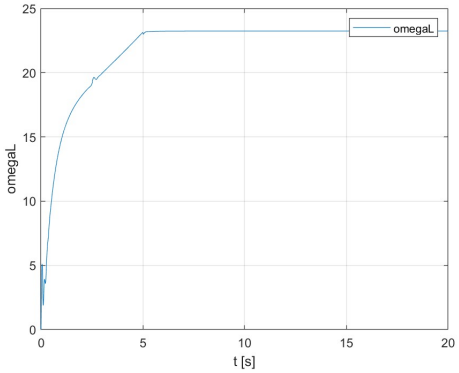
(a) Left leg joints



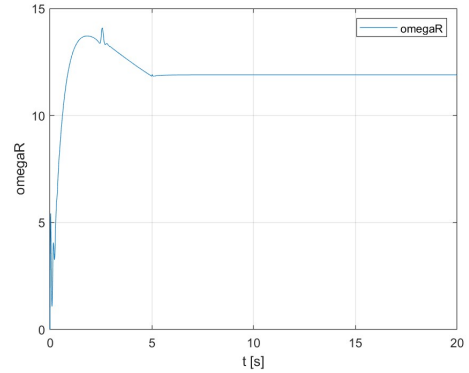
(b) Right leg joints

Figure 16: Joint trajectories during turning motion.

The wheel angular velocities behave as expected during the motion. In particular, the outer wheel exhibits a higher angular velocity than the inner one, as it travels along a larger circular path during the turning motion.



(a) Left wheel angular velocity



(b) Right wheel angular velocity

Figure 17: Wheel angular velocities during slalom motion.

The following figures 18 show the results obtained from the ODE-based numerical simulation, visualized in Simulink using the reconstructed robot model.

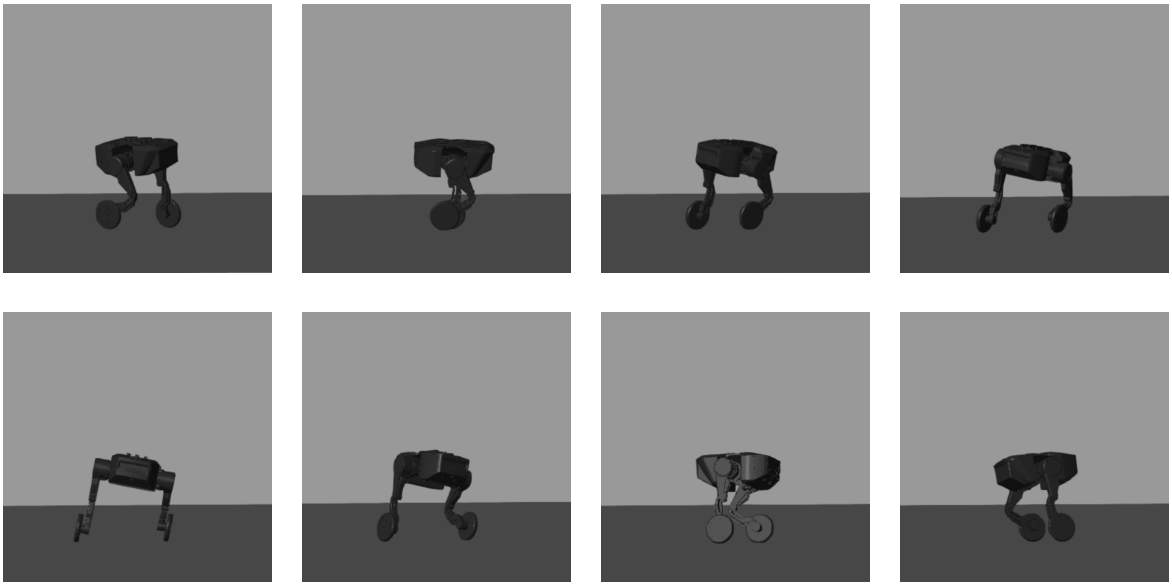


Figure 18: Sequence of frames showing the robot motion during the turning task.

Slalom Trajectory

In the third simulation, the control objective is to evaluate the performance of the robot under a more dynamic and challenging scenario. A slalom motion is generated by combining a time-varying yaw rate with a constant forward velocity. This scenario is also introduced to provide a qualitative visual validation of the robot behavior, allowing to verify that the resulting motion is physically consistent and coherent with the expected dynamics.
The desired trajectories are defined as follows:

$$\dot{\alpha}(t) = A\sin(2\pi f t), \quad \ddot{\alpha}(t) = 2\pi f A\cos(2\pi f t)$$

$$\chi^*(t) = \begin{bmatrix} v^* & 0 & z_0 & l_b\dfrac{v^*\dot{\alpha}(t)}{g} & \beta_0 & 0 \end{bmatrix}^T$$

$$\dot{\chi}^*(t) = \begin{bmatrix} 0 & 0 & 0 & l_b\dfrac{v^*}{g}\ddot{\alpha}(t) & 0 & 0 \end{bmatrix}^T$$

This combination of constant forward velocity and oscillatory yaw reference, requires continuous adaptation of the robot posture. The robot alternates left and right turning while moving forward, making the task significantly more demanding than the previous scenarios. Figure 19 shows the motion of the robot during the slalom task, where the trajectory is represented with respect to a set of reference poles, indicated in the plot as red points.

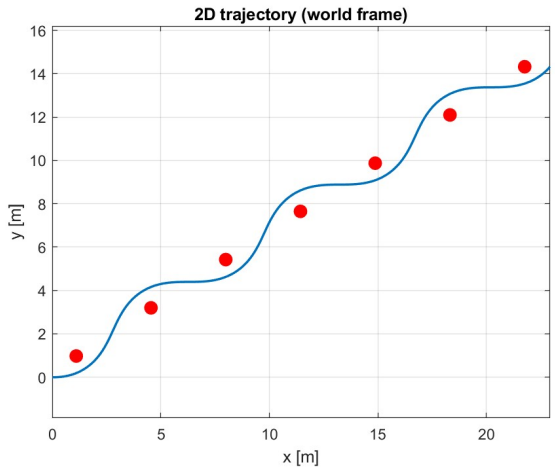


Figure 19: 2D trajectory of the robot during slalom motion with poles.

The forward velocity (Fig. 20) exhibits a fast and smooth convergence to the desired constant value, with no oscillations.

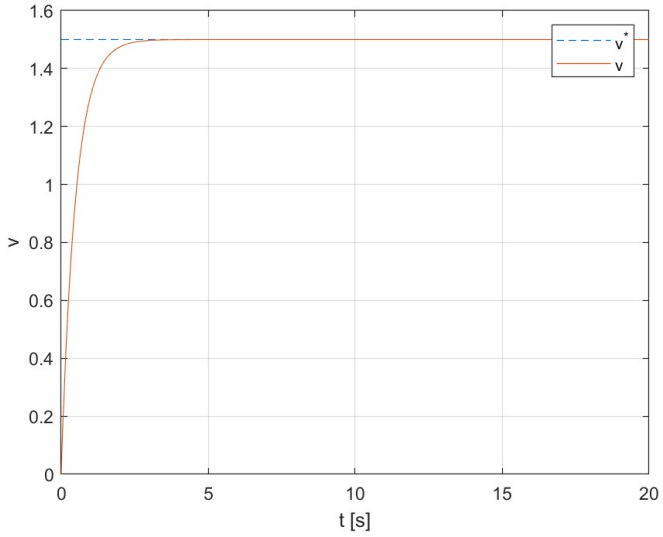
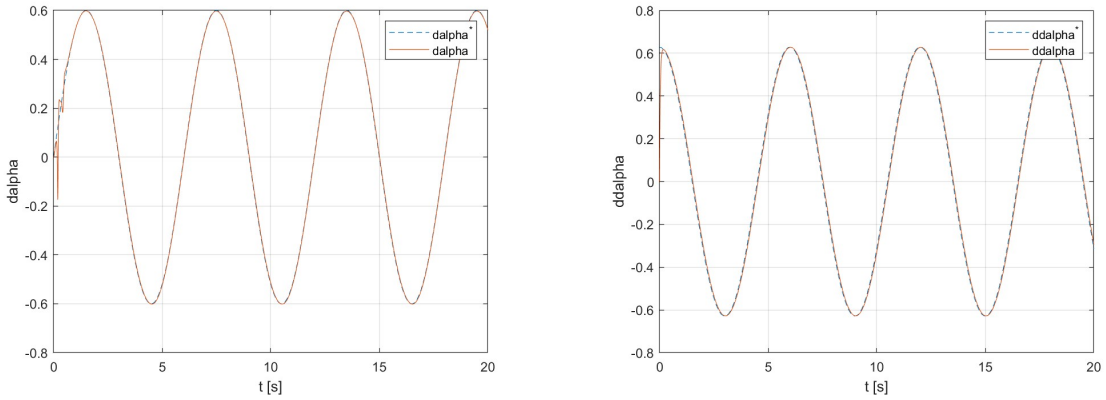


Figure 20: Forward velocity tracking.

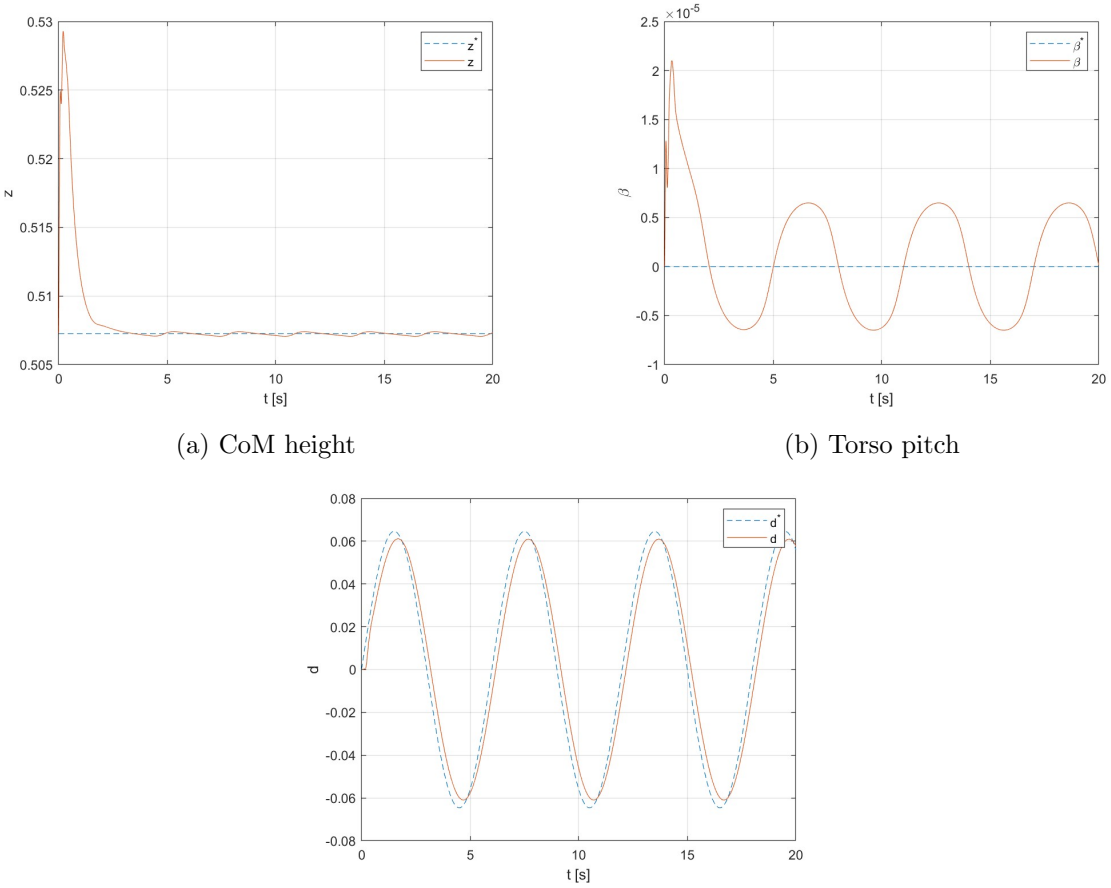
The yaw rate and yaw acceleration during the slalom motion are shown in Fig. 21. The yaw rate closely follows the sinusoidal reference, generating the alternating turning motions that characterize the slalom behavior. The corresponding yaw acceleration remains bounded and exhibits the expected oscillatory profile, confirming a smooth generation of the rotational motion.



(a) Yaw rate (b) Yaw acceleration

Figure 21: Yaw rate and yaw acceleration during slalom motion.

The CoM height, torso pitch, and leg configuration parameter during the slalom motion are shown in Fig. 22.



(a) CoM height (b) Torso pitch

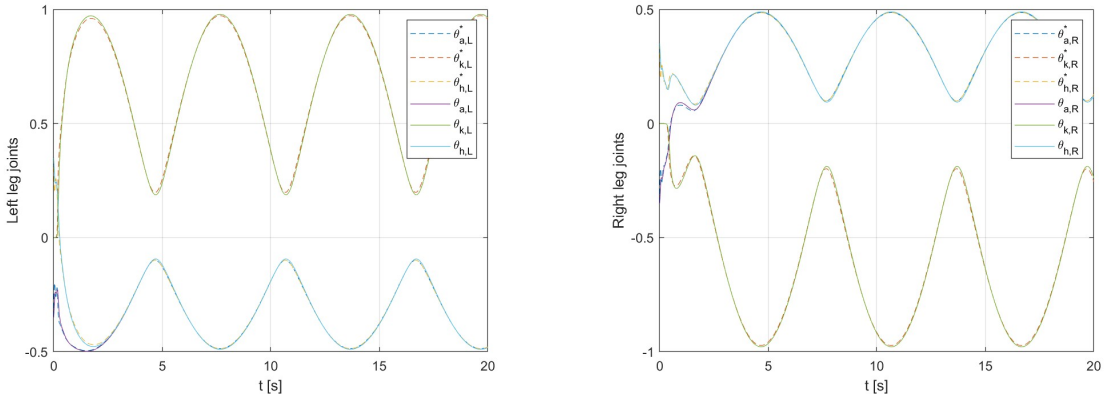
(c) Leg configuration

Figure 22: Stability-related variables during slalom motion.

Similarly to the previous tasks, these variables remain close to their desired values, confirming that balance is preserved throughout the motion. However, in this task , a

more pronounced time-varying behavior is observed.
The CoM height exhibits oscillations that follow the alternating turning motion, reflecting the continuous adaptation of the robot posture to the changing lateral dynamics (Fig.22a). The torso pitch, showed in Fig.22b, remains close to its nominal value, with small variations associated with forward motion regulation, as already observed in the previous scenarios.
The leg configuration parameter d shows a clear oscillatory pattern synchronized with the yaw rate (Fig.22c). As in the turning task, d changes sign according to the direction of rotation, generating the appropriate roll angle required to compensate for lateral acceleration. In this case, the effect is more evident due to the continuous alternation of the turning direction.

The joint trajectories are shown in Fig. 23. The joint motions exhibit coordinated but asymmetric behavior between the two legs, however, in this scenario the pattern becomes time-varying: the two legs continuously alternate their roles as the robot switches between left and right turns.
This alternating behavior follows the oscillations of the parameter d, with one leg extending while the other shortens, and vice versa.

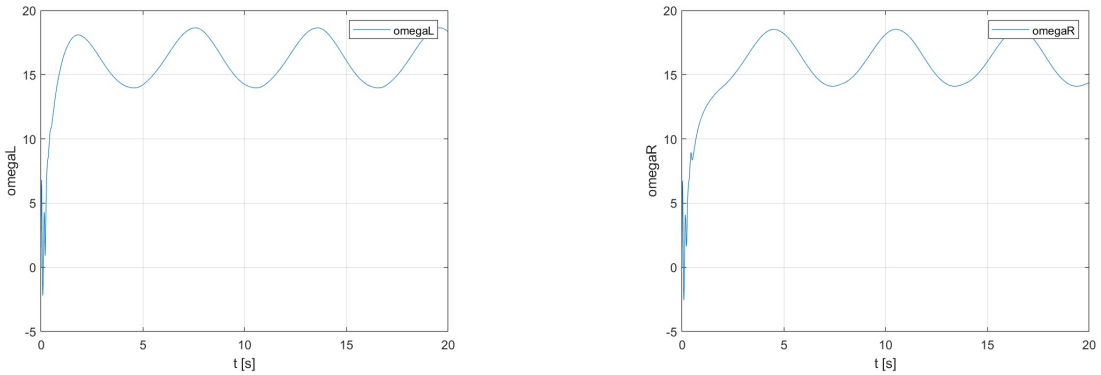


(a) Left leg joints

(b) Right leg joints

Figure 23: Joint trajectories during slalom motion.

The wheel angular velocities (Fig. 24) exhibit alternating behavior, reflecting the periodic inversion of the turning direction. The rotational motion varies continuously over time, producing the slalom trajectory.



(a) Left wheel

(b) Right wheel

Figure 24: Wheel angular velocities during slalom motion.

The following sequence of frames illustrates the robot motion during the slalom task, as obtained from the ODE-based simulation and visualized in Simulink.

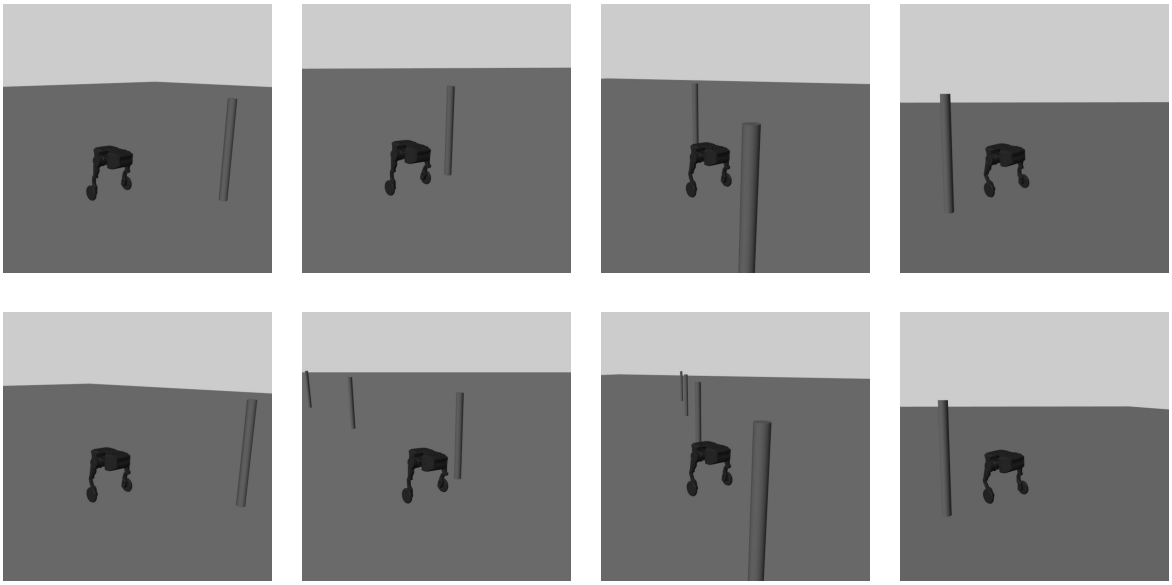


Figure 25: Sequence of frames showing the robot motion during the slalom task.

Overall, the results demonstrate that the proposed control framework is able to handle complex and highly dynamic motion tasks. Compared to the previous scenarios, the robot successfully coordinates forward motion and alternating turning while maintaining stability, confirming the robustness of the whole-body control strategy under strongly time-varying conditions.

6 Conclusions

In this work, a whole-body motion planning and control framework for a wheeled-bipedal robot has been presented. Starting from the kinematic modeling of a single wheeled leg, the relationship between joint motion and torso pose was derived and extended to the whole-body level through the formulation of the Whole-Body Jacobian. This provided a unified mapping between task space and joint space, enabling a consistent approach to motion planning and control.

The integration of a dynamic interpretation based on the wheeled inverted pendulum model allowed the control problem to be reformulated in terms of CoM posture regulation. Within this framework, forward motion and balance are achieved indirectly by controlling the horizontal displacement of the CoM, rather than directly commanding wheel rotation. This leads to a control architecture that combines task-space planning, joint-space tracking, and wheel actuation in a coherent and structured way.

The proposed approach demonstrates that a variety of motions, including forward locomotion and turning maneuvers, can be achieved while maintaining stability through coordinated whole-body behavior. The simulation results confirm the expected system behavior and validate the effectiveness of the proposed control framework in achieving stable and consistent motion.

References

[1] Yu Wang, Yaxian Xin, Xuewen Rong, and Yibin Li. Whole-body motion planning and control for underactuated wheeled-bipdal robots. In *2021 IEEE International Conference on Robotics and Biomimetics (ROBIO)*, pages 1071–1076, 2021.